

# SOLID Principles

## Introduction

**SOLID** is a set of five object-oriented design principles that help developers build software that is maintainable, scalable, flexible, and easy to test. These principles were introduced by **Robert C. Martin**, commonly known as **Uncle Bob**, and are considered the foundation of clean software architecture.

SOLID principles encourage developers to write code that is:

- Easy to understand
- Easy to modify
- Easy to test
- Loosely coupled
- Highly cohesive
- Extensible without modifying existing code

Modern .NET applications, including **ASP.NET Core**, **Entity Framework Core**, **Minimal APIs**, and **Microservices**, rely heavily on SOLID principles to achieve clean architecture and maintainability.

---

## What is SOLID?

SOLID is an acronym representing five design principles.

| Letter | Principle                       |
|--------|---------------------------------|
| S      | Single Responsibility Principle |
| O      | Open/Closed Principle           |
| L      | Liskov Substitutio              |

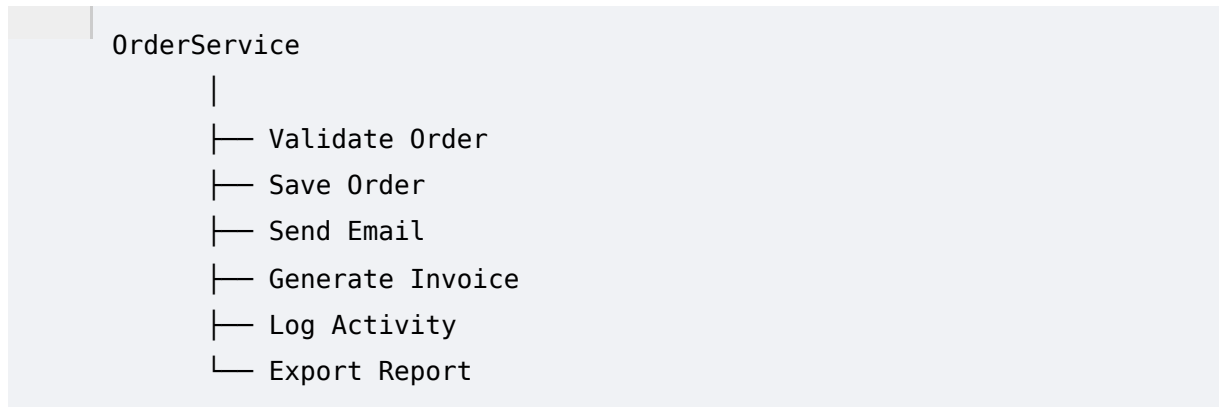
|   | n Principle                     |
|---|---------------------------------|
| I | Interface Segregation Principle |
| D | Dependency Inversion Principle  |

Each principle addresses a specific aspect of software design, but together they promote loosely coupled, maintainable systems.

---

## Why SOLID?

Imagine an application where one class performs multiple unrelated tasks.



Problems:

- Difficult to understand
- Difficult to modify
- Difficult to test
- High coupling
- Frequent code changes
- Increased risk of introducing bugs

SOLID encourages separating these responsibilities into focused classes.

```
OrderService
|
├─ ValidationService
├─ Repository
├─ EmailService
├─ InvoiceService
└─ Logger
```

This design is cleaner, more modular, and easier to maintain.

---

## S — Single Responsibility Principle (SRP)

### Definition

**A class should have only one reason to change.**

In other words, a class should have only one responsibility.

---

### Bad Example

```
public class Employee
{
    public void CalculateSalary()
    {
    }

    public void SaveToDatabase()
    {
    }

    public void SendEmail()
    {
    }

    public void GenerateReport()
    {
    }
}
```

Problems:

- Multiple responsibilities
- Hard to maintain
- Difficult to test

- Changes in one feature may affect others

---

## Good Example

```
public class Employee
{
    public decimal CalculateSalary()
    {
        return 50000;
    }
}

public class EmployeeRepository
{
    public void Save(Employee employee)
    {
    }
}

public class EmailService
{
    public void SendEmail(Employee employee)
```

Each class now has a single responsibility.

---

## Benefits of SRP

- Easier maintenance
- Better readability
- Easier testing
- Smaller classes
- Reduced coupling

---

## O — Open/Closed Principle (OCP)

### Definition

**Software entities should be open for extension but closed for modification.**

This means new functionality should be added without changing existing code.

---

## Bad Example

```
public class DiscountService
{
    public decimal Calculate(string customerType)
    {
        if(customerType == "Regular")
            return 5;

        if(customerType == "Premium")
            return 10;

        return 0;
    }
}
```

Adding a new customer type requires modifying the existing class.

---

### Good Example

```
public interface IDiscount
{
    decimal Calculate();
}

public class RegularCustomer : IDiscount
{
    public decimal Calculate() => 5;
}

public class PremiumCustomer : IDiscount
{
    public decimal Calculate() => 10;
}
```

Adding a VIP customer only requires creating another implementation.

```
public class VipCustomer : IDiscount
{
    public decimal Calculate() => 20;
}
```

No existing code changes.

---

## Benefits of OCP

- Easier extension
  - Lower risk of bugs
  - Better maintainability
  - Supports plugin architectures
- 

## L — Liskov Substitution Principle (LSP)

### Definition

**Derived classes should be replaceable with their base classes without changing the correctness of the program.**

Objects of child classes should behave as expected when used through the parent type.

---

### Bad Example

```
public class Bird
{
    public virtual void Fly()
    {
    }
}

public class Penguin : Bird
{
    public override void Fly()
    {
        throw new NotImplementedException();
    }
}
```

Penguins cannot fly, violating the expected behavior of `Bird`.

---

### Good Example

```
public abstract class Bird
{
}

public interface IFlyingBird
```

```

{
    void Fly();
}

public class Sparrow : Bird, IFlyingBird
{
    public void Fly()
    {
    }
}

public class Penguin : Bird

```

Only birds that can fly implement `IFlyingBird` .

---

### Benefits of LSP

- Reliable inheritance
  - Better polymorphism
  - Fewer runtime errors
  - Improved design consistency
- 

## I — Interface Segregation Principle (ISP)

### Definition

**Clients should not be forced to depend on interfaces they do not use.**

Large interfaces should be split into smaller, focused interfaces.

---

### Bad Example

```

public interface IWorker
{
    void Work();

    void Eat();

    void Sleep();
}

```

A robot worker doesn't need `Eat()` or `Sleep()` .

---

## Good Example

```
public interface IWork
{
    void Work();
}

public interface IEat
{
    void Eat();
}

public interface ISleep
{
    void Sleep();
}
```

Implement only what is needed.

```
public class Robot : IWork
{
    public void Work()
    {
    }
}
```

---

## Benefits of ISP

- Smaller interfaces
  - Easier implementation
  - Less unnecessary code
  - Better flexibility
- 

## D — Dependency Inversion Principle (DIP)

### Definition

**High-level modules should not depend on low-level modules. Both should depend on abstractions.**

Additionally:

- Abstractions should not depend on details.



- Details should depend on abstractions.

This principle forms the foundation of **Dependency Injection**.

---

### Bad Example

```
public class EmailService
{
    public void Send()
    {
    }
}

public class OrderService
{
    private readonly EmailService _email =
        new EmailService();
}
```

Problems:

- Tight coupling
  - Difficult to test
  - Cannot replace EmailService easily
- 

### Good Example

```
public interface IEmailService
{
    void Send();
}

public class EmailService : IEmailService
{
    public void Send()
    {
    }
}

public class OrderService
{
    private readonly IEmailService _email;
```

```
public OrderService(IEmailService email)
```

Register using Dependency Injection.

```
builder.Services.AddScoped<IEmailService, EmailService>();
```

Now `OrderService` depends on an abstraction rather than a concrete implementation.

---

### Benefits of DIP

- Loose coupling
- Easier unit testing
- Better flexibility
- Easier replacement of implementations
- Supports Dependency Injection

---

## SOLID in ASP.NET Core

A typical ASP.NET Core application naturally follows SOLID principles.



Examples:

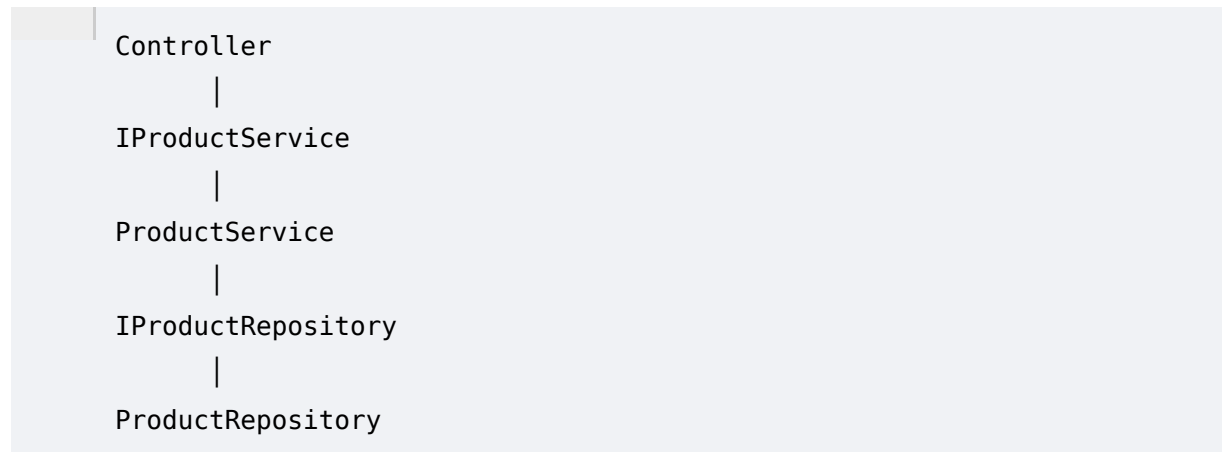
- Controllers have a single responsibility (SRP).
- Services implement interfaces and can be extended (OCP).
- Interfaces allow substituting implementations (LSP).
- Small service interfaces follow ISP.
- Dependency Injection implements DIP.

---

## SOLID and Dependency Injection

Dependency Injection complements SOLID.

Example:



Each layer depends on interfaces rather than concrete classes, making the application modular and testable.

---

## Advantages of SOLID Principles

- Improved code readability
  - Easier maintenance
  - Better scalability
  - Loose coupling
  - High cohesion
  - Easier testing
  - Better extensibility
  - Reduced code duplication
  - Lower risk of regression bugs
  - Cleaner architecture
- 

## Common Mistakes

### Overusing Interfaces

Not every class requires an interface. Creating interfaces for simple, unlikely-to-change classes can introduce unnecessary complexity.

---

### Large Service Classes

Avoid creating "God Classes" that contain many unrelated responsibilities.

Example:

```
OrderService
```

```
1000+ Lines
```

```
50 Methods
```

Instead, split functionality into focused services.

---

### Tight Coupling

Avoid directly instantiating dependencies.

Bad:

```
var email = new EmailService();
```

Good:

```
public OrderService(IEmailService email)
{
}
```

### Misusing Inheritance

Inheritance should model an "is-a" relationship. If classes do not naturally fit this relationship, prefer composition over inheritance.

---

## Best Practices

- Give each class a single responsibility.
  - Prefer composition over inheritance where appropriate.
  - Program against interfaces rather than implementations.
  - Use Dependency Injection.
  - Keep interfaces small and focused.
  - Write unit tests for business logic.
  - Avoid duplicate code.
  - Separate business logic from infrastructure concerns.
  - Refactor large classes into smaller, cohesive components.
  - Apply SOLID principles consistently rather than selectively.
- 

## SOLID in a Real-World E-Commerce Application



Each layer has a clearly defined responsibility, depends on abstractions, and can be independently tested or replaced without affecting other layers.

---

## SOLID Principles Summary

| Principle                   | Meaning                                | Main Goal     |
|-----------------------------|----------------------------------------|---------------|
| Single Responsibility (SRP) | One class, one responsibility          | High cohesion |
| Open/Closed (OCP)           | Extend without modifying existing code | Extensibility |

|                             |                                                        |                             |
|-----------------------------|--------------------------------------------------------|-----------------------------|
| Liskov Substitution (LSP)   | Derived classes should be replaceable for base classes | Reliable inheritance        |
| Interface Segregation (ISP) | Prefer small, focused interfaces                       | Reduced coupling            |
| Dependency Inversion (DIP)  | Depend on abstractions, not concrete implementations   | Flexibility and testability |

---

## SOLID vs Non-SOLID Design

| Aspect          | Without SOLID | With SOLID |
|-----------------|---------------|------------|
| Coupling        | High          | Low        |
| Maintainability | Difficult     | Easy       |
| Testing         | Difficult     | Simple     |
| Scalability     | Limited       | High       |

|               |         |           |
|---------------|---------|-----------|
| Reusability   | Low     | High      |
| Extensibility | Poor    | Excellent |
| Readability   | Complex | Clear     |
| Risk of Bugs  | Higher  | Lower     |

---

## Interview Questions

### 1. What does SOLID stand for?

SOLID stands for Single Responsibility, Open/Closed, Liskov Substitution, Interface Segregation, and Dependency Inversion.

---

### 2. Which SOLID principle is most closely related to Dependency Injection?

The **Dependency Inversion Principle (DIP)**. Dependency Injection is a practical implementation of DIP, allowing classes to depend on abstractions instead of concrete implementations.

---

### 3. Which principle helps avoid "God Classes"?

The **Single Responsibility Principle (SRP)**.

---

### 4. Why is the Open/Closed Principle important?

It allows applications to evolve by adding new functionality through extensions rather than modifying existing, tested code, reducing the risk of introducing bugs.

---

### 5. Which SOLID principle improves unit testing the most?

While all principles contribute, the **Dependency Inversion Principle (DIP)** has the greatest impact because dependencies can be mocked or substituted during testing.

---

## Summary

SOLID principles provide a foundation for designing robust, maintainable, and extensible object-oriented software. By ensuring that classes have focused

responsibilities, depend on abstractions, expose only necessary interfaces, and support safe extension and substitution, developers can build applications that are easier to understand, test, and evolve. In the .NET ecosystem, SOLID principles work hand in hand with Dependency Injection, ASP.NET Core, Entity Framework Core, and Clean Architecture, making them essential knowledge for building enterprise-grade applications.